

Students Project

Virtual Machine Firewall Implementation

Understanding Protocols, Ports, Firewall Policies & Rule Implementation

Course: Computer
CYS2313

Estimated Time: **7-14**

Students:

احمد علام
اسامه حسنين
محمد حابس
نايف الحارثي
خالد العمودي

Network Operations

hours

1. Project Overview

In this hands-on lab, you will set up a Linux virtual machine from scratch, then configure a host-based firewall to control incoming and outgoing network traffic. The project is designed to give you practical experience with the concepts that underpin every enterprise network: protocols, ports, firewall policies, and rule-based packet filtering.

By the end of this lab you will have a working firewall that enforces a real-world security policy you designed yourself.

2. Learning Objectives

After completing this project, you will be able to:

- Explain the role of common network protocols (TCP, UDP, ICMP) and their associated port numbers.
- Differentiate between well-known ports (0–1023), registered ports (1024–49151), and dynamic ports (49152–65535).
- Describe what a firewall policy is and why a default-deny approach is preferred over default-allow.
- Use both UFW and iptables to create, modify, and verify firewall rules on a Linux system.
- Test firewall rules using standard network diagnostic tools (ping, curl, nmap, netcat).
- Document a firewall policy and justify each rule based on security requirements.

3. Background Knowledge

Review the following concepts before you begin. You will need this understanding to complete the tasks.

3.1 Protocols

A protocol is a set of rules that governs how data is transmitted across a network. The three protocols you will work with most in this lab are:

Protocol	Description	Common Use Cases
TCP	Connection-oriented; guarantees delivery via a three-way handshake.	HTTP/HTTPS (web), SSH (remote access), FTP (file transfer), SMTP (email).
UDP	Connectionless; faster but no delivery guarantee.	DNS (name resolution), DHCP (IP assignment), streaming media, VoIP.
ICMP	Used for diagnostics and error reporting; not a transport protocol.	ping (echo request/reply), traceroute, destination unreachable messages.

3.2 Ports

A port number identifies a specific service on a host. Think of the IP address as a street address and the port as the apartment number.

Range	Category	Examples
0 – 1023	Well-Known Ports	22 (SSH), 53 (DNS), 80 (HTTP), 443 (HTTPS)

1024 – 49151	Registered Ports	3306 (MySQL), 5432 (PostgreSQL), 8080 (alt HTTP)
49152 – 65535	Dynamic / Ephemeral	Assigned temporarily by the OS for client-side connections.

3.3 Firewall Policy

A firewall policy is a high-level document that defines what traffic should be allowed and what should be blocked. A good policy follows the principle of least privilege: deny everything by default and only permit traffic that is explicitly needed.

Every firewall rule typically specifies:

- Direction: Inbound (incoming) or Outbound (outgoing).
- Protocol: TCP, UDP, or ICMP.
- Port(s): The destination port number or range.
- Source / Destination: IP address or subnet.
- Action: ALLOW or DENY.

3.4 UFW vs. iptables

UFW (Uncomplicated Firewall) is a user-friendly front-end for iptables. It simplifies common tasks but hides some of the underlying power. iptables gives you full control over the Linux netfilter framework. In this lab you will use both so you can see how they relate to each other.

4. Prerequisites

- A computer with at least 4 GB of RAM and 20 GB of free disk space.
- A hypervisor installed. VirtualBox (free, recommended) or VMware Workstation Player — you may use any hypervisor you are comfortable with.
- An Ubuntu Server 22.04 LTS ISO (download from ubuntu.com).
- Basic familiarity with the Linux command line (cd, ls, nano/vim, sudo).

5. Project Tasks

Part A — Set Up Your Virtual Machine

1. Create a new VM in your hypervisor with the following minimum specs: 1 CPU, 2 GB RAM, 20 GB disk, bridged or NAT network adapter.
2. Install Ubuntu Server 22.04 LTS. During installation, enable the OpenSSH server when prompted.
3. After installation, log in and run the following commands to update the system:

```
sudo apt update && sudo apt upgrade -y
```

4. Install the tools you will need for testing:

```
sudo apt install -y nmap netcat-openbsd curl net-tools
```

5. Verify your network is working:

```
ip addr show          # Note your VM's IP address
ping -c 3 8.8.8.8     # Test internet connectivity
ping -c 3 google.com  # Test DNS resolution
```

Deliverable A: Take a screenshot showing your VM's IP address and successful ping results.

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:8d:17:06 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c298d1706
    inet 192.168.142.130/24 metric 100 brd 192.168.142.255 scope global dynamic ens33
        valid_lft 1393sec preferred_lft 1393sec
    inet6 fe80::20c:29ff:fe8d:1706/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
PING google.com (142.251.36.206) 56(84) bytes of data.
64 bytes from ncmsb-an-in-f14.1e100.net (142.251.36.206): icmp_seq=1 ttl=128 time=76.6 ms
64 bytes from ncmsb-an-in-f14.1e100.net (142.251.36.206): icmp_seq=2 ttl=128 time=76.5 ms
64 bytes from ncmsb-an-in-f14.1e100.net (142.251.36.206): icmp_seq=3 ttl=128 time=76.9 ms

--- google.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 76.496/76.661/76.921/0.186 ms

```

Part B — Design Your Firewall Policy

Before touching any firewall commands, write a policy document. Use the table below as a template. Your policy must include at least six rules that cover the following scenarios:

- Allow SSH access (port 22) from your host machine only.
- Allow HTTP (port 80) and HTTPS (port 443) inbound for a web server.
- Allow DNS queries (port 53, UDP) outbound.
- Block all other inbound traffic (default deny).
- Allow established and related connections (stateful filtering).
- One custom rule of your choice (e.g., allow MySQL from a specific subnet, block ICMP, rate-limit SSH).

Policy Template:

#	Direction	Protocol	Port(s)	Source / Dest	Action	Justification
1	Inbound	TCP	22	192.168.1.0/24	ALLOW	SSH admin access
2	Inbound	TCP	80, 443	Any	ALLOW	Web server traffic
3	Outbound	UDP	53	Any	ALLOW	Allow DNS resolution for system updates.
4	Inbound	Any	Any	Any	DENY	Strict default-deny to protect all other ports.
5	Inbound	TCP/UDP	Any	Any	ALLOW	Stateful filtering for ongoing connections.
6	Inbound	ICMP	N/A	Any	DENY	Block Ping requests to hide server from discovery.

Deliverable B: A completed policy table with at least six rules and a justification for each.

Part C — Implement with UFW (Beginner-Friendly)

UFW is the easiest way to get started. Implement your policy using the commands below as a guide.

1. Reset UFW to a clean state and set default policies:

```
sudo ufw reset
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

2. Add your rules one at a time. Examples:

```
# Allow SSH from your local network only
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp
```

```
# Allow HTTP and HTTPS from anywhere
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

```
# Allow outbound DNS
sudo ufw allow out 53/udp
```

3. Enable the firewall and verify:

```
sudo ufw enable
sudo ufw status verbose
```

Deliverable C: A screenshot of the output of “ufw status verbose” showing all your rules active.

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN 192.168.142.0/24
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
3306/tcp ALLOW IN 192.168.142.0/24
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)

53/udp ALLOW OUT Anywhere
53/udp (v6) ALLOW OUT Anywhere (v6)
```

Part D — Implement with iptables (Advanced)

Now replicate the same policy using raw iptables commands. This gives you direct visibility into how the Linux kernel filters packets.

1. Disable UFW so it does not conflict:

```
sudo ufw disable
```

2. Flush all existing rules and set default chain policies:

```
sudo iptables -F
sudo iptables -X
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT
```

3. Allow loopback traffic (essential for local services):

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

```
sudo iptables -A OUTPUT -o lo -j ACCEPT
```

4. Allow established and related connections (stateful rule):

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

5. Add your specific rules. Examples:

```
# Allow SSH from local network
```

```
sudo iptables -A INPUT -p tcp -s 192.168.1.0/24 --dport 22 -j ACCEPT
```

```
# Allow HTTP and HTTPS
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
```

```
# Allow outbound DNS
```

```
sudo iptables -A OUTPUT -p udp --dport 53 -j ACCEPT
```

6. View your complete ruleset:

```
sudo iptables -L -v -n --line-numbers
```

Deliverable D: A screenshot of the full iptables listing with line numbers, plus your iptables commands saved to a script file.

Part E — Test Your Firewall

Testing is critical. For each rule, verify that allowed traffic passes and blocked traffic is dropped.

What to Test	Command (from host or second VM)	Expected Result
SSH access (allowed)	<code>ssh user@<VM-IP></code>	Connection succeeds
HTTP access (allowed)	<code>curl http://<VM-IP></code>	Response received (or connection refused if no web server)
Blocked port (e.g., 3306)	<code>nmap -p 3306 <VM-IP></code>	Port shown as filtered/closed
ICMP ping (depends on your policy)	<code>ping <VM-IP></code>	Replies if allowed; timeout if blocked
Full port scan	<code>nmap -sS <VM-IP></code>	Only allowed ports appear as open

Deliverable E: A testing log that shows the command, its output, and whether the result matches your policy (pass/fail) for each rule.

Part F — Reflection & Documentation

Write a short report (1–2 pages) that answers the following questions:

- What is the difference between a default-deny and a default-allow firewall policy? Which is more secure and why?
- Compare UFW and iptables: which did you find easier to use? In what scenario would iptables be the better choice?
- Pick one of your rules and explain how it would protect against a specific attack scenario.
- What would you add to your firewall policy if this were a production server?

Deliverable F: Your written reflection as a PDF or Word document or Power Point.

. Security Policies: Deny vs. Allow

- **The Difference:** A **Default-Allow** policy permits all traffic by default, only blocking known threats (Blacklisting), while **Default-Deny** blocks everything except explicitly authorized connections (Whitelisting).
- **Security Verdict:** The **Default-Deny** approach is superior because it adheres to the **Principle of Least Privilege**. It minimizes the attack surface by ensuring that "shadow" services or unknown vulnerabilities remain unreachable to outsiders.

2. Comparison: UFW vs. iptables

- **Ease of Use:** **UFW** is the preferred choice for fast, user-friendly configuration due to its intuitive command syntax.
- **Iptables Strength:** **iptables** is better for complex, high-traffic environments. It provides **granular control** over packet filtering and is necessary for advanced tasks like stateful inspection, NAT, or isolating specific network interfaces.

3. Security Justification: Web Traffic Rule (Ports 80/443)

- **The Scenario:** This rule allows inbound HTTP/HTTPS traffic to the web server.
- **Protection Logic:** By specifically permitting TCP traffic on these ports, the firewall automatically drops unauthorized protocols (like UDP or ICMP) directed at the web server. This prevents attackers from using open ports to probe the system or launch non-web-based exploits.

4. Production Recommendations

To harden a live production server, I would implement:

- **Web Application Firewall (WAF):** To filter malicious HTTP requests like SQL injections.
- **Intrusion Prevention (Fail2Ban):** To automatically block IPs showing suspicious behavior.
- **Rate Limiting:** To mitigate Denial of Service (DoS) attacks by limiting connection frequency.
- **Active Logging:** To monitor and audit dropped packets for forensic analysis.

6. Grading Rubric

Criteria	Points	Description
VM Setup (Part A)	10	VM is running, updated, and has working network connectivity.
Firewall Policy (Part B)	20	Policy table has 6+ rules, proper use of direction/protocol/port fields, and meaningful justifications.
UFW Implementation (Part C)	15	All policy rules correctly implemented in UFW. Default deny is set.
iptables Implementation (Part D)	20	Equivalent rules in iptables. Includes stateful rule, loopback, and correct chain policies.
Testing (Part E)	20	Each rule tested with appropriate tool. Results documented with pass/fail and explanation.
Reflection (Part F)	15	Thoughtful analysis of FW concepts, tool comparison, and real-world application.
Total	100	

7. Submission Checklist

Before submitting, make sure you have:

- Screenshot of VM IP address and successful ping (Part A).
- Completed firewall policy table with 6+ rules and justifications (Part B).
- Screenshot of UFW status verbose output (Part C).
- Screenshot of iptables listing and your iptables script file (Part D).
- Testing log with commands, outputs, and pass/fail results for each rule (Part E).
- Reflection report answering all four questions (Part F).

8. Helpful Resources

- Ubuntu UFW documentation: help.ubuntu.com/community/UFW
- iptables tutorial: netfilter.org/documentation
- Common ports reference: iana.org/assignments/service-names-port-numbers
- nmap basics: nmap.org/book/man.html

Good luck.... build it, break it, learn from it!